

PB007: Software Engineering

Lecture 3: Agile & Software Process Models

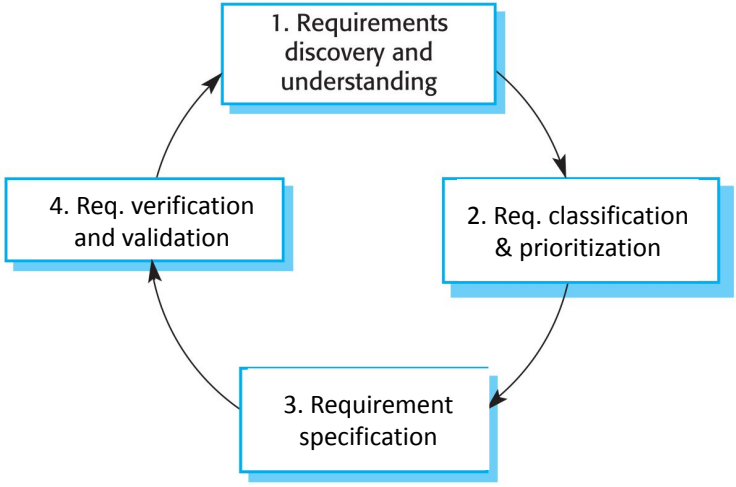
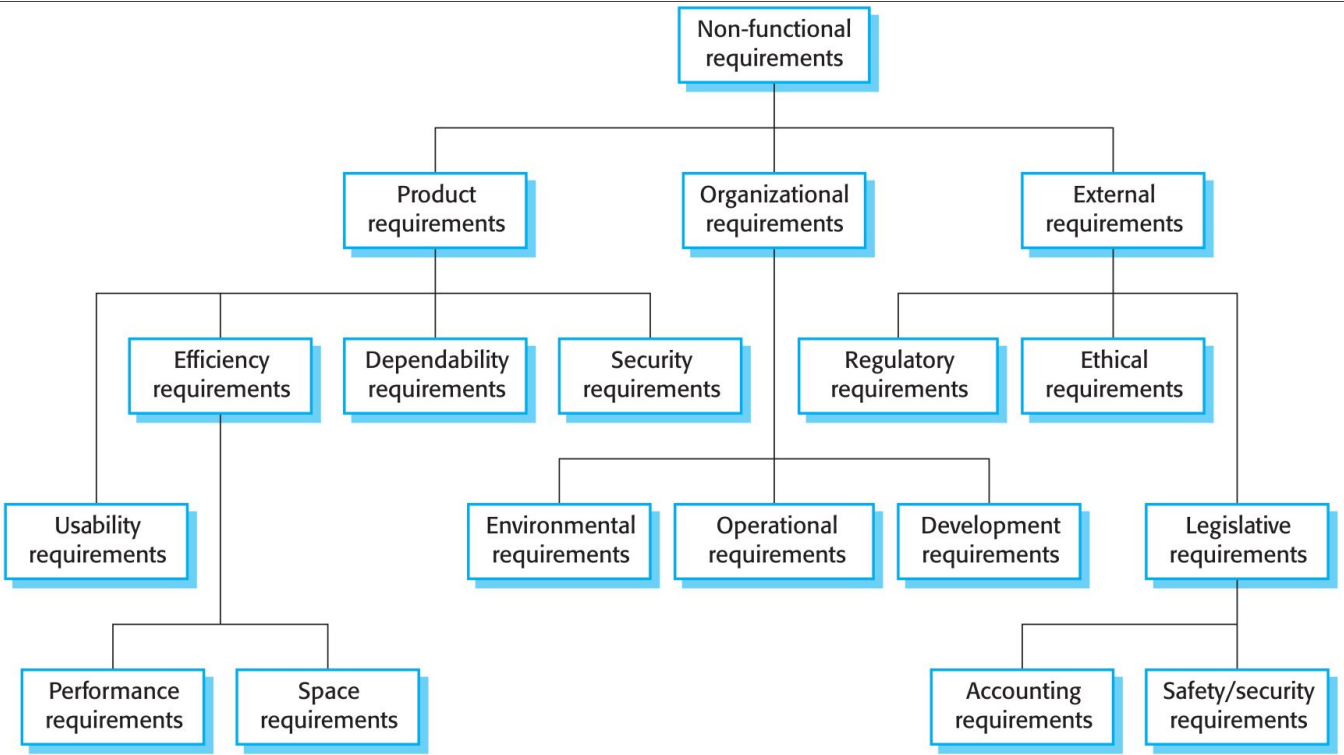
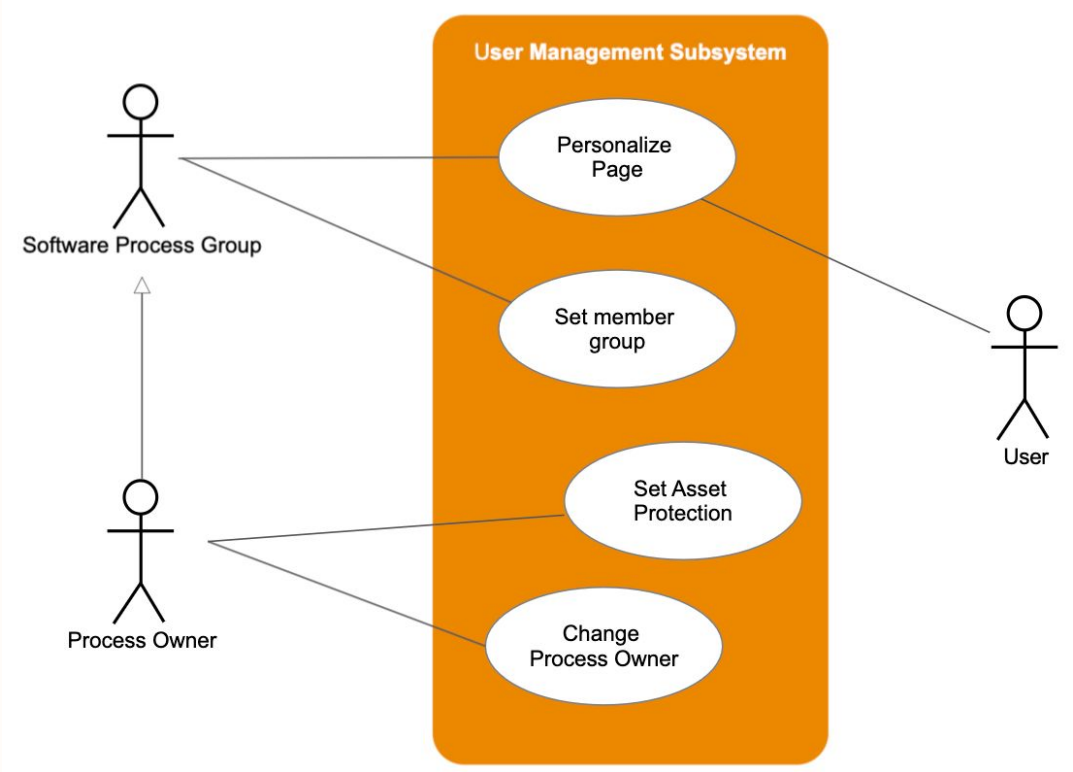
Stanislav Chren

Autumn 2026

Slido.com
#PB007



Recap



#	Knowledge Area
✓ 1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Outline

1. **Software Process Models**
2. **Agile Development**
3. **Agile Practices**

1. Software Process Models

Software Development as Coordinated Work

- Software projects involve many connected activities
- Requirements influence design
- Design influences implementation
- Testing may reveal problems in earlier decisions

Process models help us reason about coordination, feedback, and control



Software Process

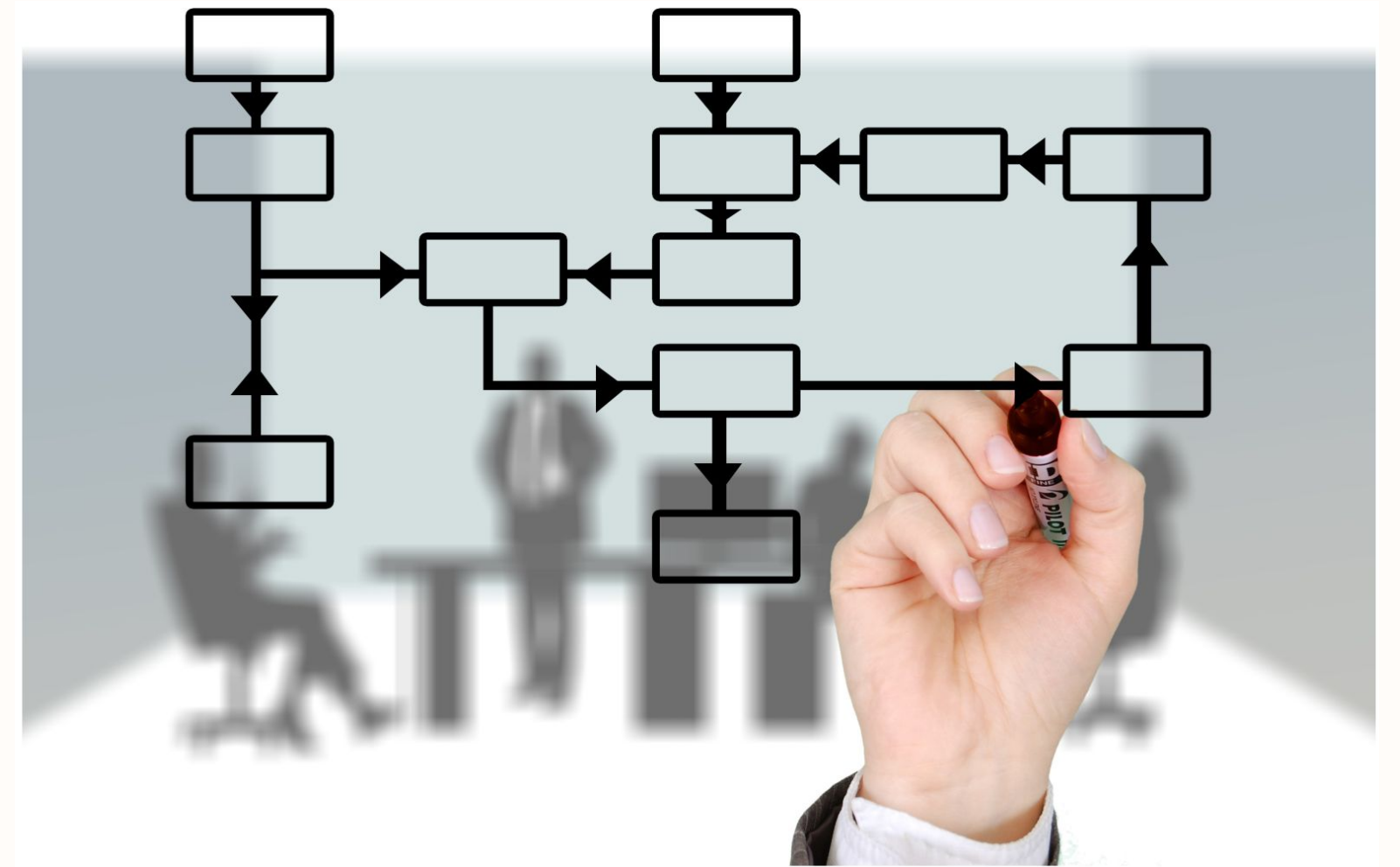
- The way an organisation, team, or individual carries out software engineering
- May be formally documented, or informally agreed
- Every organisation has a process, even if it is not written down
- Organisations may:
 - adopt a known framework
 - tailor a known framework
 - define their own process



Software Process

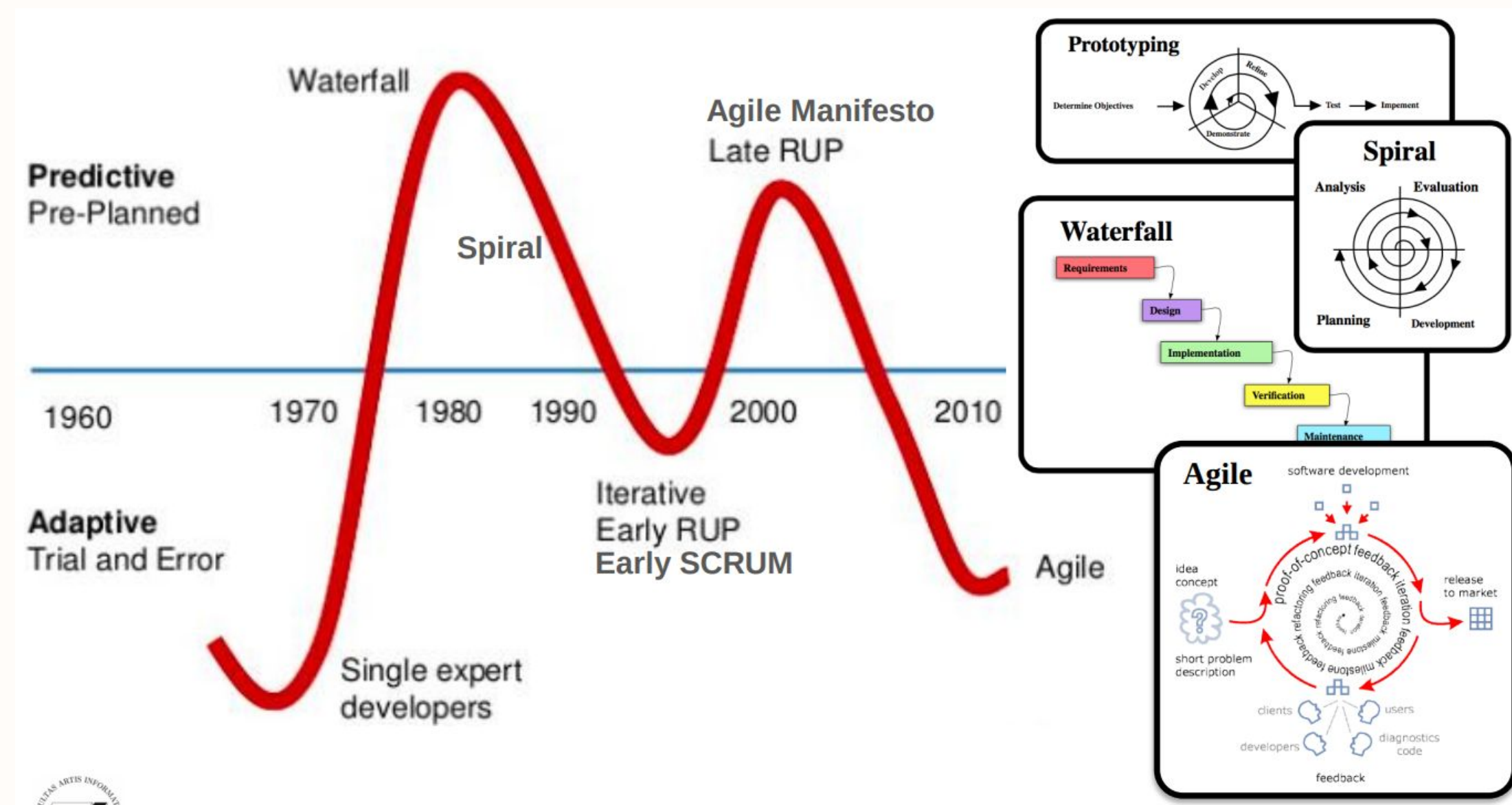
Software process description usually contains:

- **Activities**
 - **what is done**
 - **requirements, design, coding, testing, deployment**
- **Artifacts**
 - **what is produced or used**
 - **specifications, code, tests, manuals**
- **Actors/roles**
 - **who is involved**
 - **developers, testers, users, product owners**



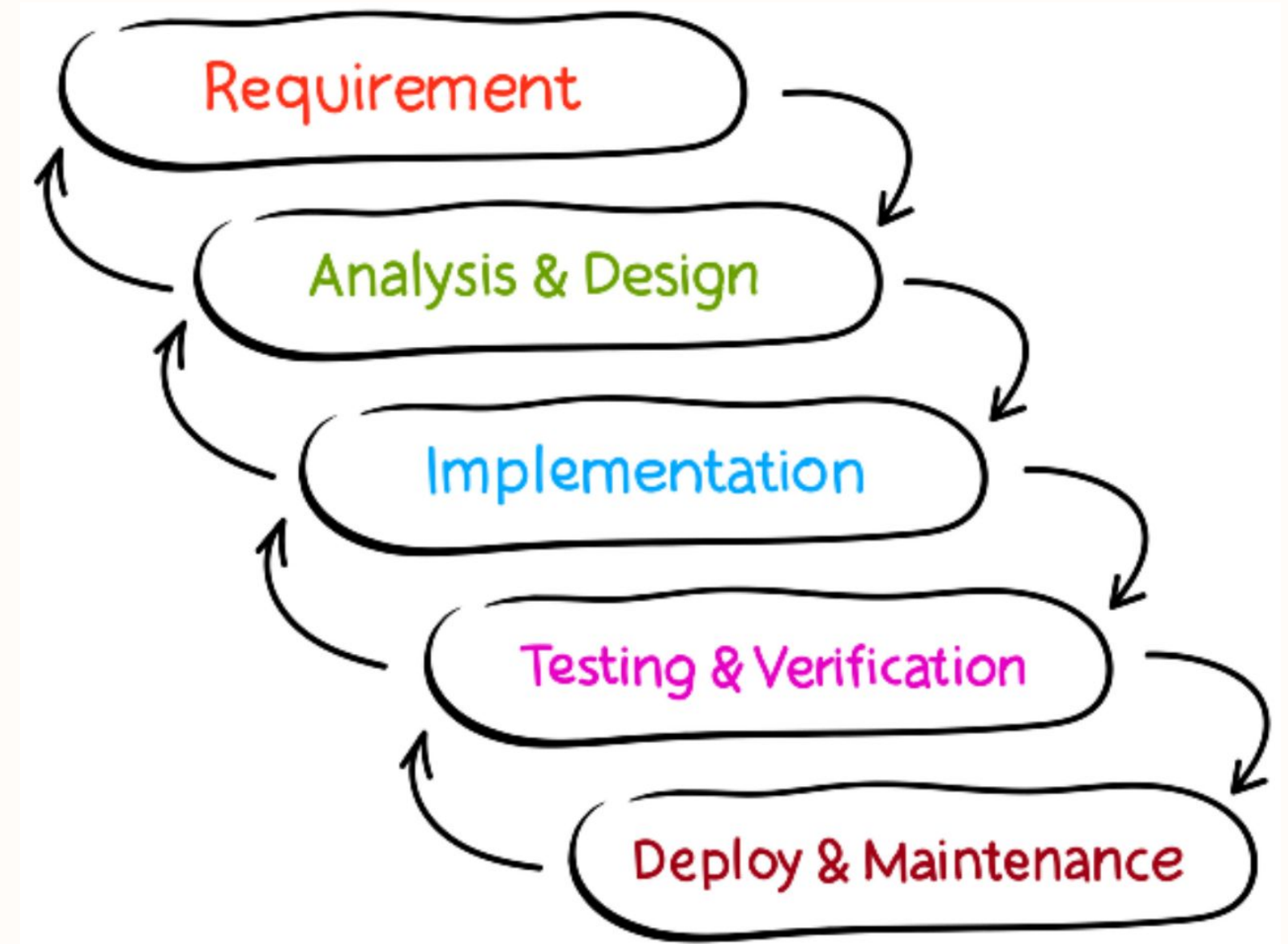
Software Life-Cycle Models

- High-level models **of the software process**
- **Show the main** activities of software engineering
- **Emphasize** how activities are ordered **over time**
- **Typical activities**
 - requirements
 - design
 - implementation
 - testing
 - deployment
 - maintenance
 - retirement



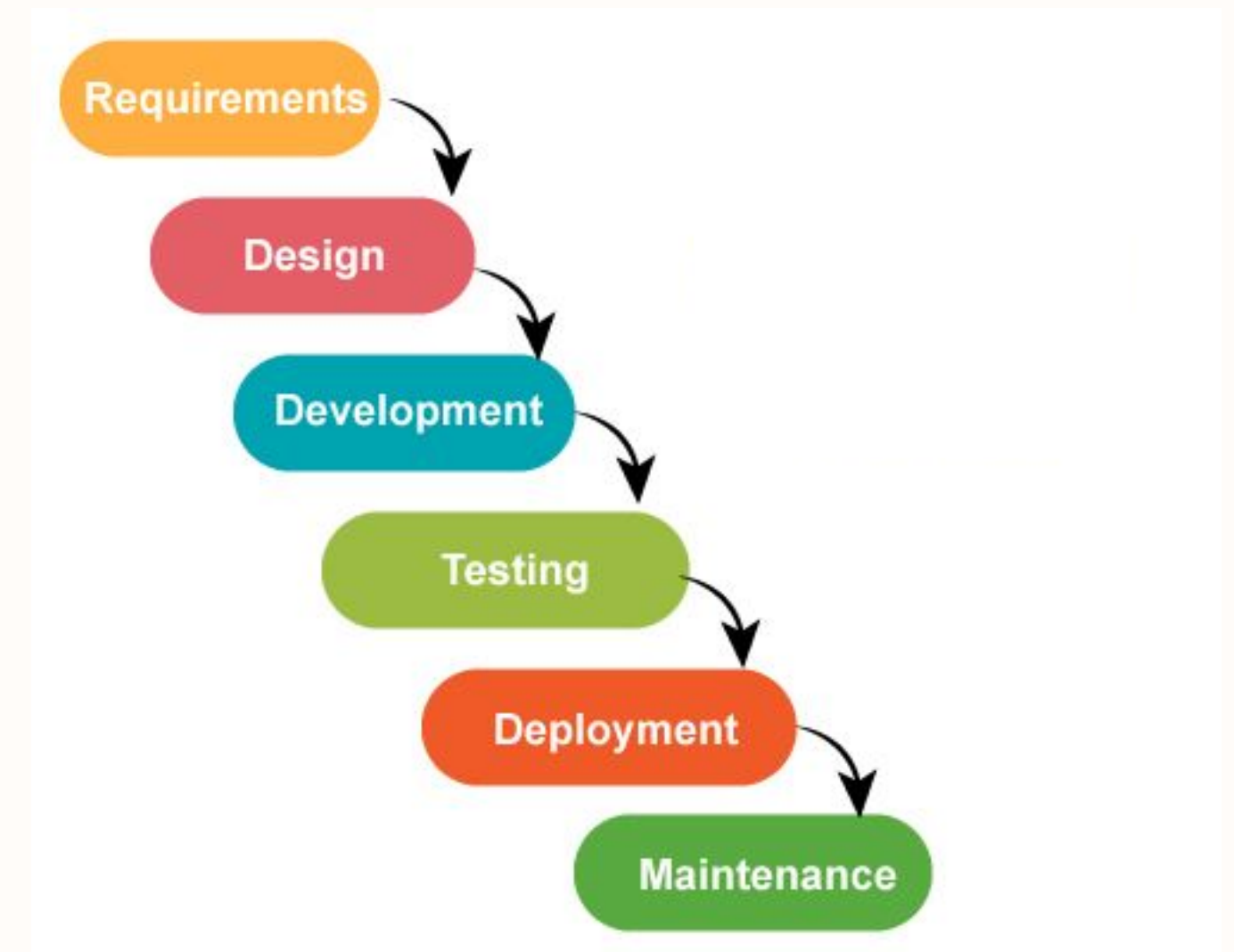
Sequential Life-Cycle Model

- Activities are organised as **phases**
- Each phase is completed before the next one starts
- Documents are reviewed, approved **and** frozen
- Changes go through formal change management
- Strong **emphasis on** planning **and** control



The Waterfall Model

- One of the most influential models in software engineering
- Associated with early software engineering discussions around 1970
- **How it works**
 - Define and approve requirements
 - Analyse and design the system
 - Implement the system
 - Integrate and test the complete system
 - Deploy the system
 - Maintain the system after delivery
- **Key assumption**
 - If early phases are done carefully enough, later rework can be minimised



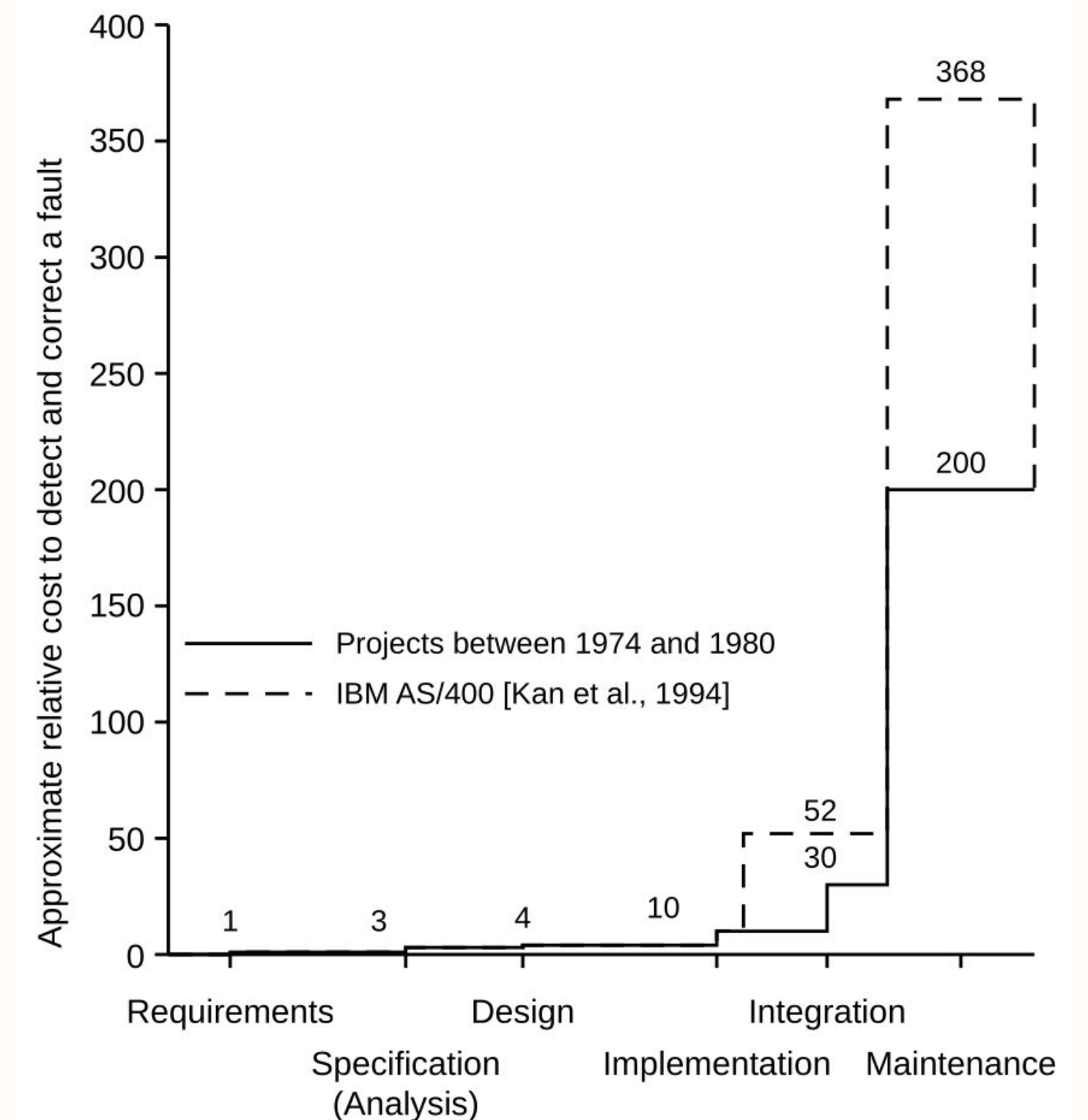
The Waterfall Model - Pros & Cons



- Easy to understand and explain
- Clear milestones and deliverables
- Extensive documentation and traceability
- Fits many contract and procurement models
- Can work when requirements are stable and well understood



- Assumes requirements can be known early
- Feedback from users arrives late
- Integration and performance problems may appear late
- Late changes can require updates to many documents and artifacts
- Can become expensive when uncertainty is high



The Waterfall Model - When to Use?

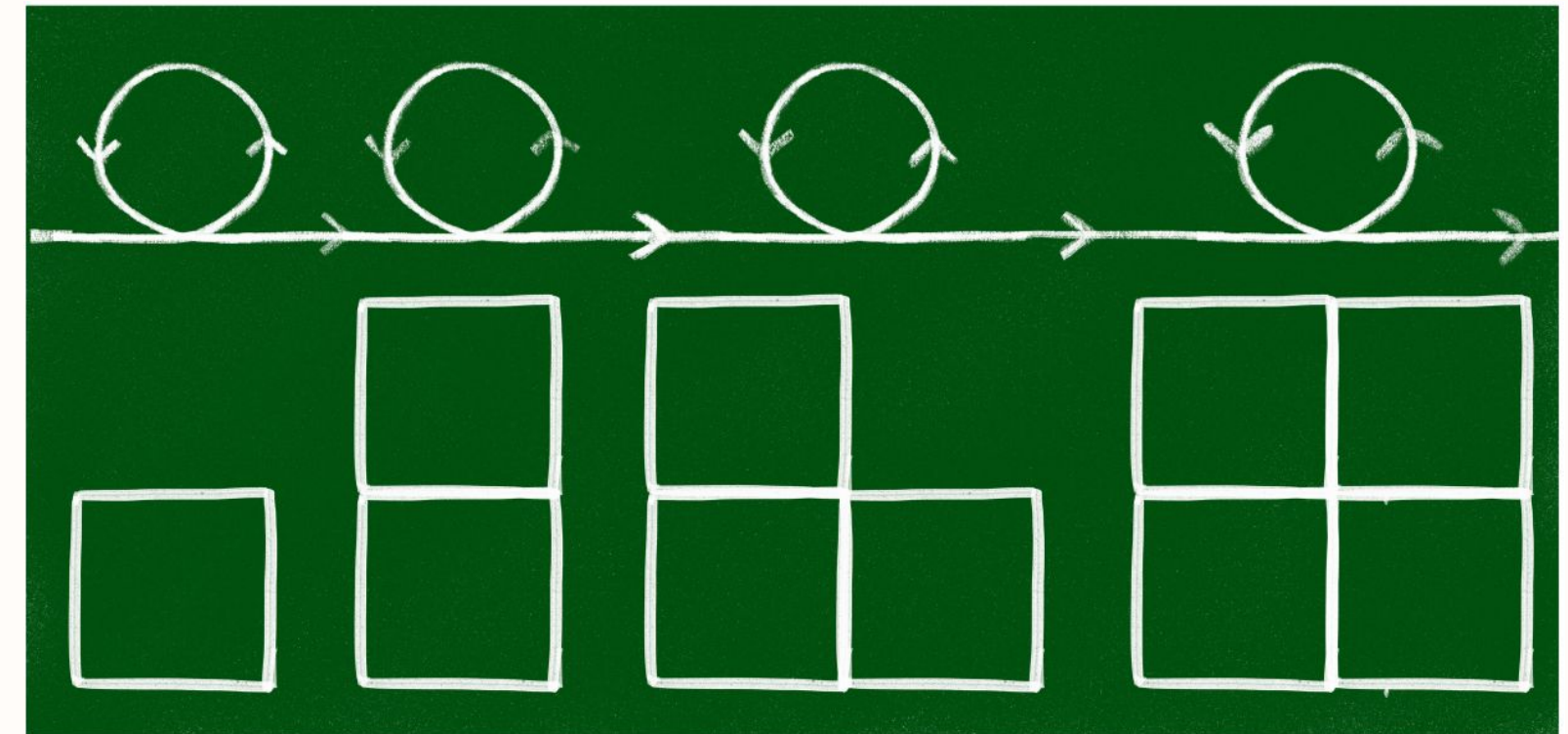
- Requirements are well understood
- Expected changes are limited
- Strong documentation is required
- Formal approval gates are important
- The customer or regulator requires phase-based control



The model is historically old, but still relevant in many organisations and contracts

Iterative and Incremental Development

- **Divide the project into increments**
- **Each** increment adds functionality
- **Each iteration is a** development cycle
- **Each iteration produces** stable, integrated, tested software
- **The** final iteration **produces the** complete product



Iterative

The team repeats development cycles: requirements, design, implementation, testing

Review happen more than once

Incremental

The product grows over time by adding useful pieces of functionality

An Iteration as a Small Project

- Clarify selected requirements
- Design relevant solution
- Implement the selected functionality
- Integrate with the existing system
- Test the increment
- Review results and collect feedback



Iterative/Incremental Development - Pros & Cons



- Customer value can be delivered earlier
- Stakeholders can give feedback on working software
- Early increments can help reveal real requirements
- Performance and integration issues can appear earlier
- Lower risk of building the wrong system
- Tested software becomes a meaningful progress indicator

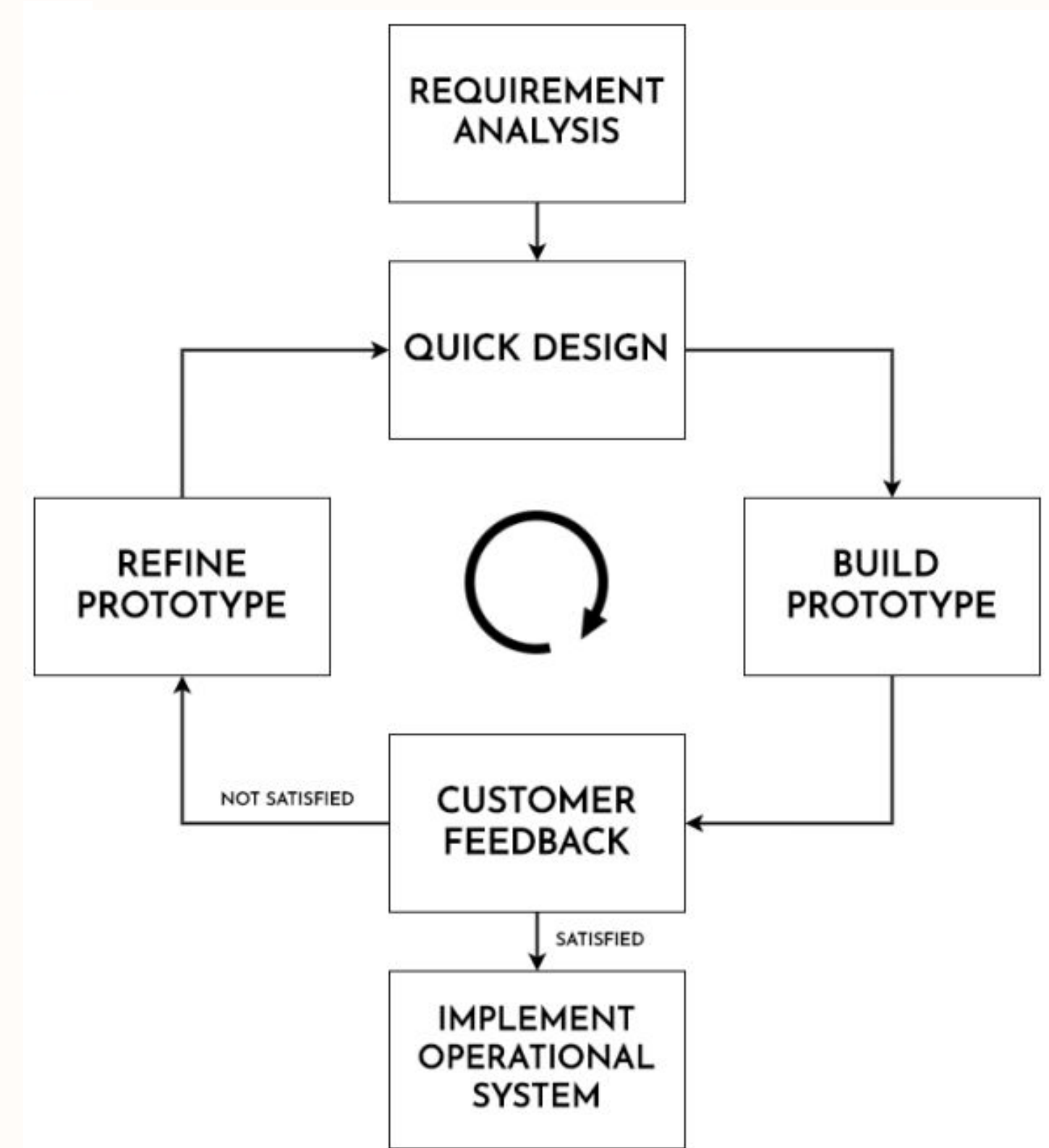


- Can be harder to plan and control
- Requires repeated planning and repeated testing
- May require more experienced staff
- Architecture must support change
- Contract models may need to be more flexible



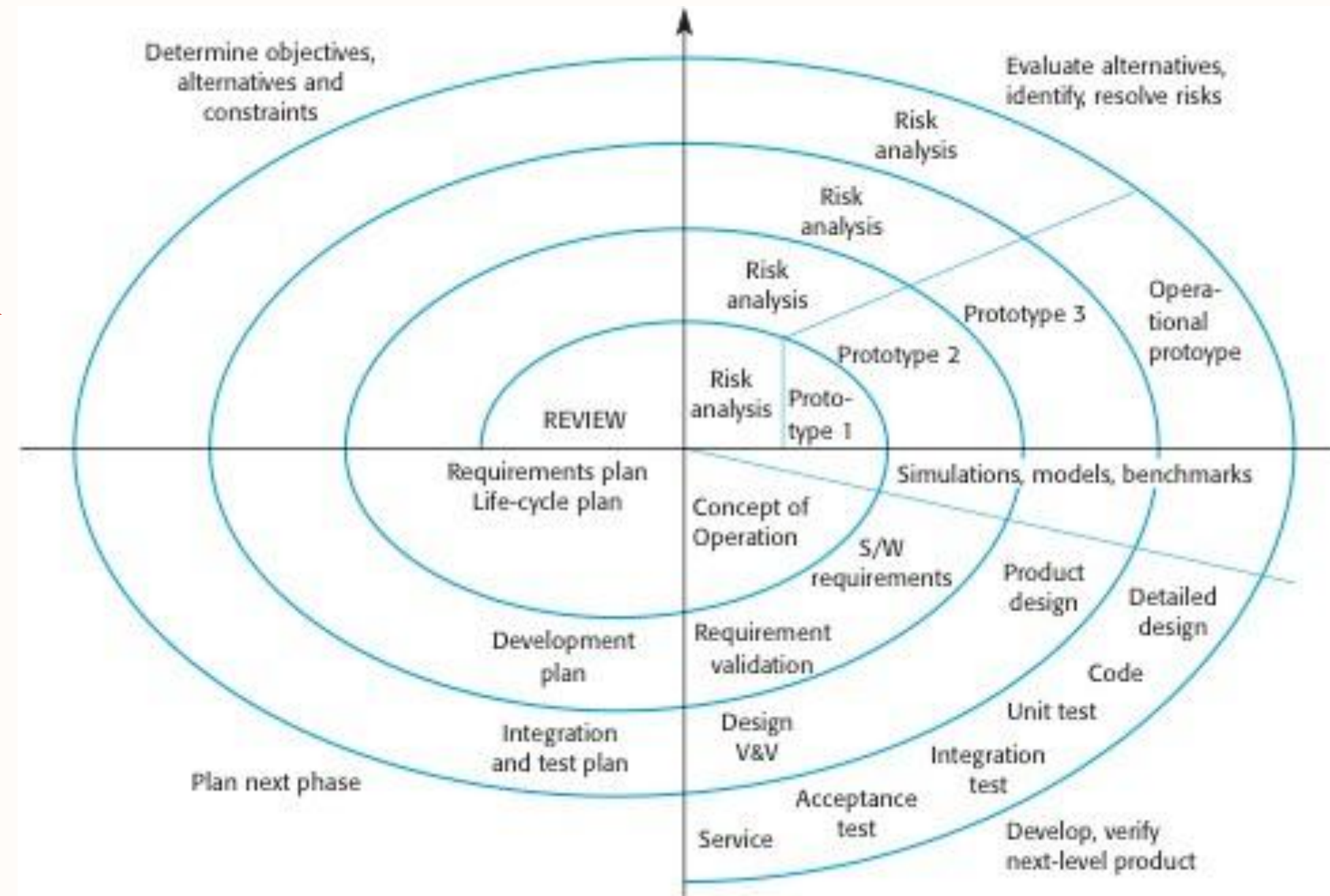
Software prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- A prototype can be used in:
 - The requirements engineering process to help with requirements elicitation, consistency checking and validation;
 - In design processes to explore design options and develop a UI design;
- Prototypes often have poor internal structure and thus should not become the foundation of the final system.



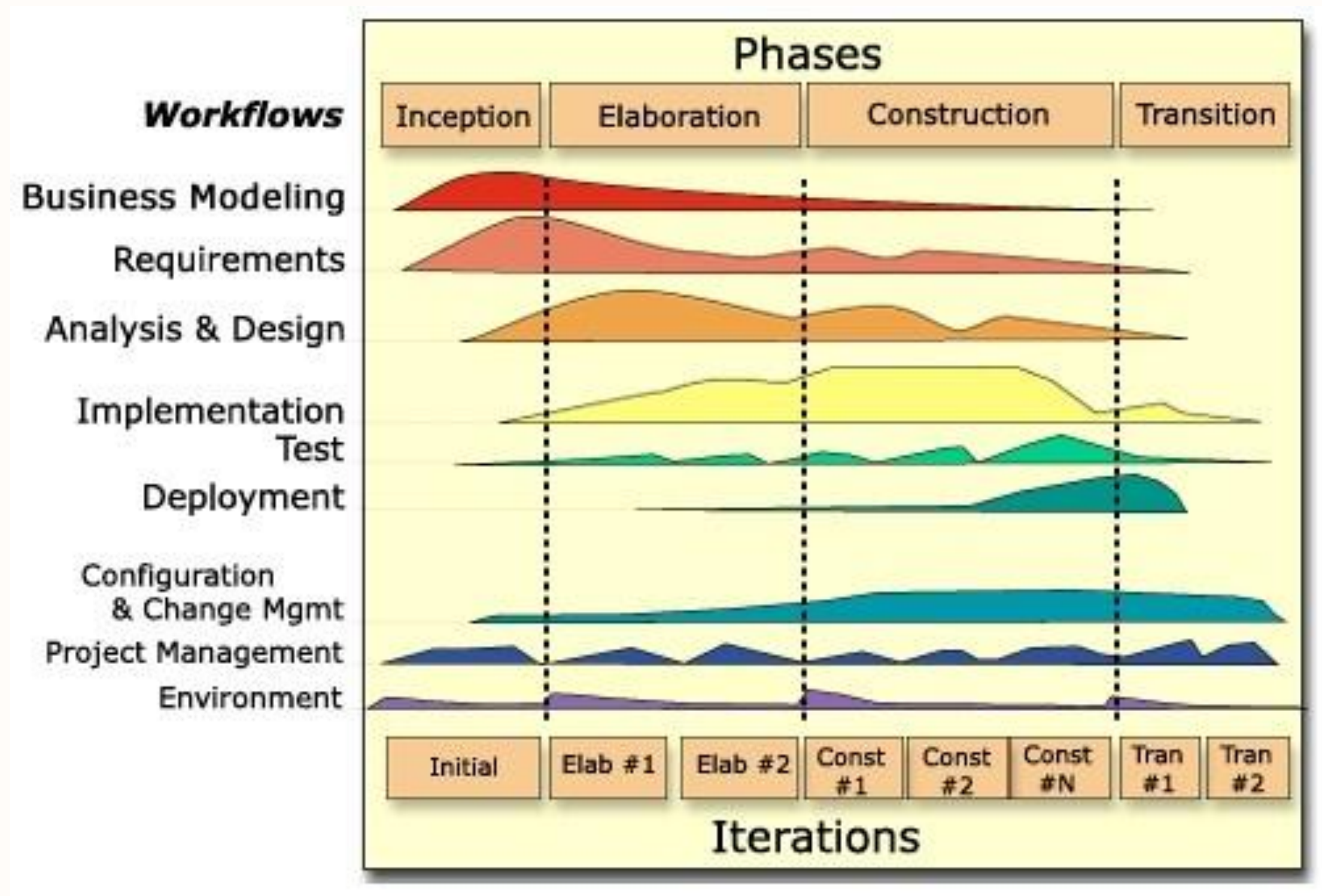
Boehm's spiral model

- Process is represented as a spiral rather than as a sequence of activities with backtracking.
- Each loop in the spiral represents a phase in the process.
- No fixed phases such as specification or design - loops in the spiral are chosen depending on what is required.
- Risks are explicitly assessed and resolved throughout the process.



Rational Unified Process (RUP)

- Developed by Rational Corporation; later associated with IBM
- A modern generic process commonly associated with the Unified Modeling Language (UML).
- Combines waterfall-like phases with iterative development
- Iterative and incremental, but often document-heavy



Microsoft Synchronise-and-Stabilise

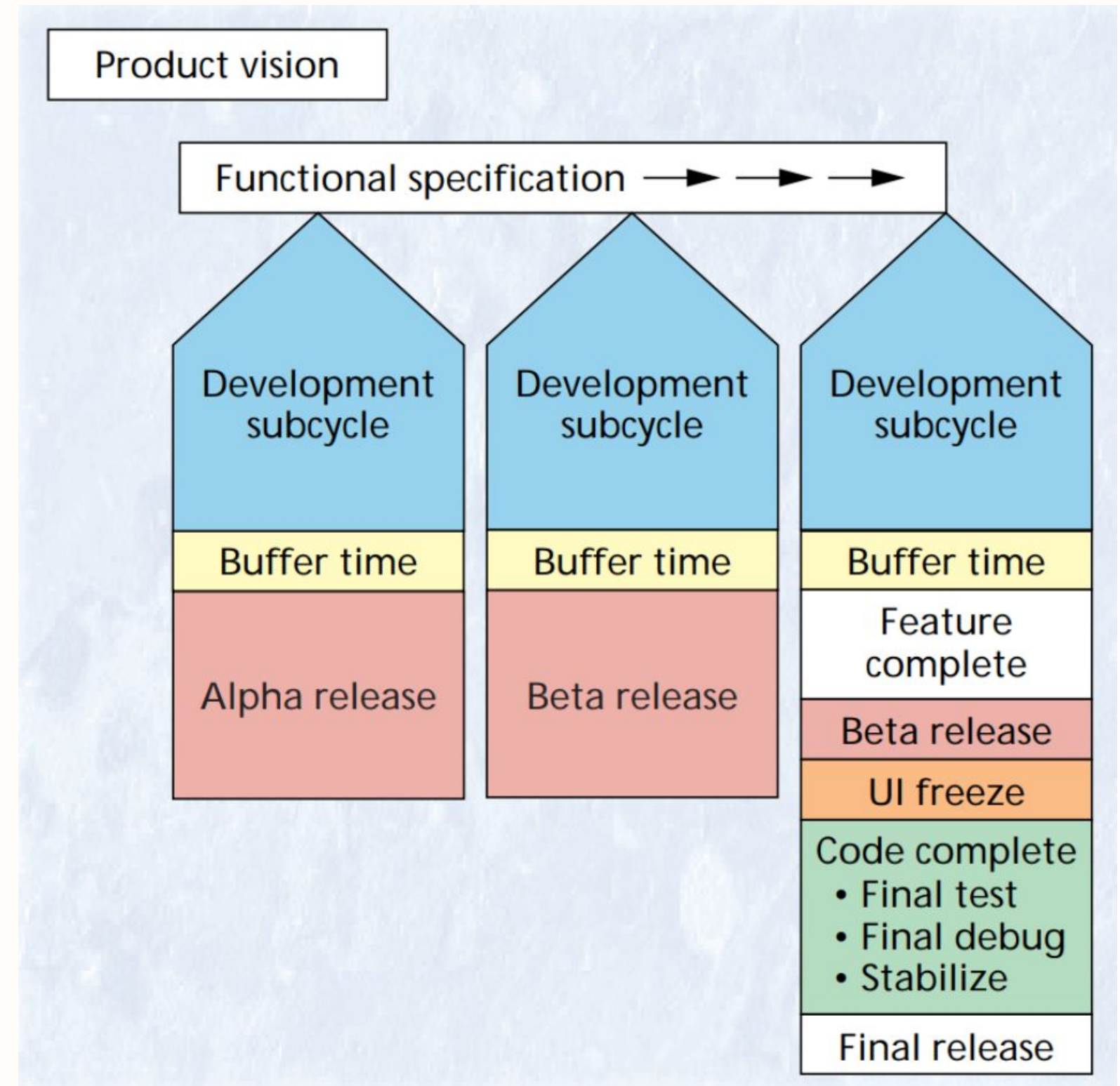
- Associated with Microsoft Office development in the 1990s
- Example of a company-specific process
- Starts with a product vision and high-level specification
- Divides development into a small number of builds
- Teams work in parallel

Synchronise

- integrate work regularly
- build the system
- run tests
- precursor to modern continuous integration

Stabilise

- restrict new feature work
- focus on testing, debugging, and fixing the build



Reuse-oriented software engineering

- ✧ **Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems.**
- ✧ **Process stages**
 - **Component analysis;**
 - **Requirements modification;**
 - **System design with reuse;**
 - **Development and integration.**
- ✧ **Reuse is now the standard approach for building many types of business system**

AI-assisted Software Engineering

- Supports task such as:
 - code completion and generation
 - test generation
 - documentation and code explanation
 - review and repair suggestions
- Evidence suggests benefits for some well-specified tasks
- Generated artefacts may be incorrect, insecure, or contextually unsuitable
- Human review, testing, and validation remain essential

Key points

- **General process models describe the organization of software processes.**
- **Process models make assumptions about uncertainty, feedback, risk and control**
- **Over time, processes started to include activities to cope with change**
- **The process models have their benefits and drawbacks**

1. Agile Development

Agile Manifesto

Individuals and Interactions	over	Processes and Tools
Working Product	over	Comprehensive Documentation
Customer Collaboration	over	Contract Negotiation
Responding to Change	over	Following a Plan

*That is, while there is value in the items on the right,
we value the items on the left more.*

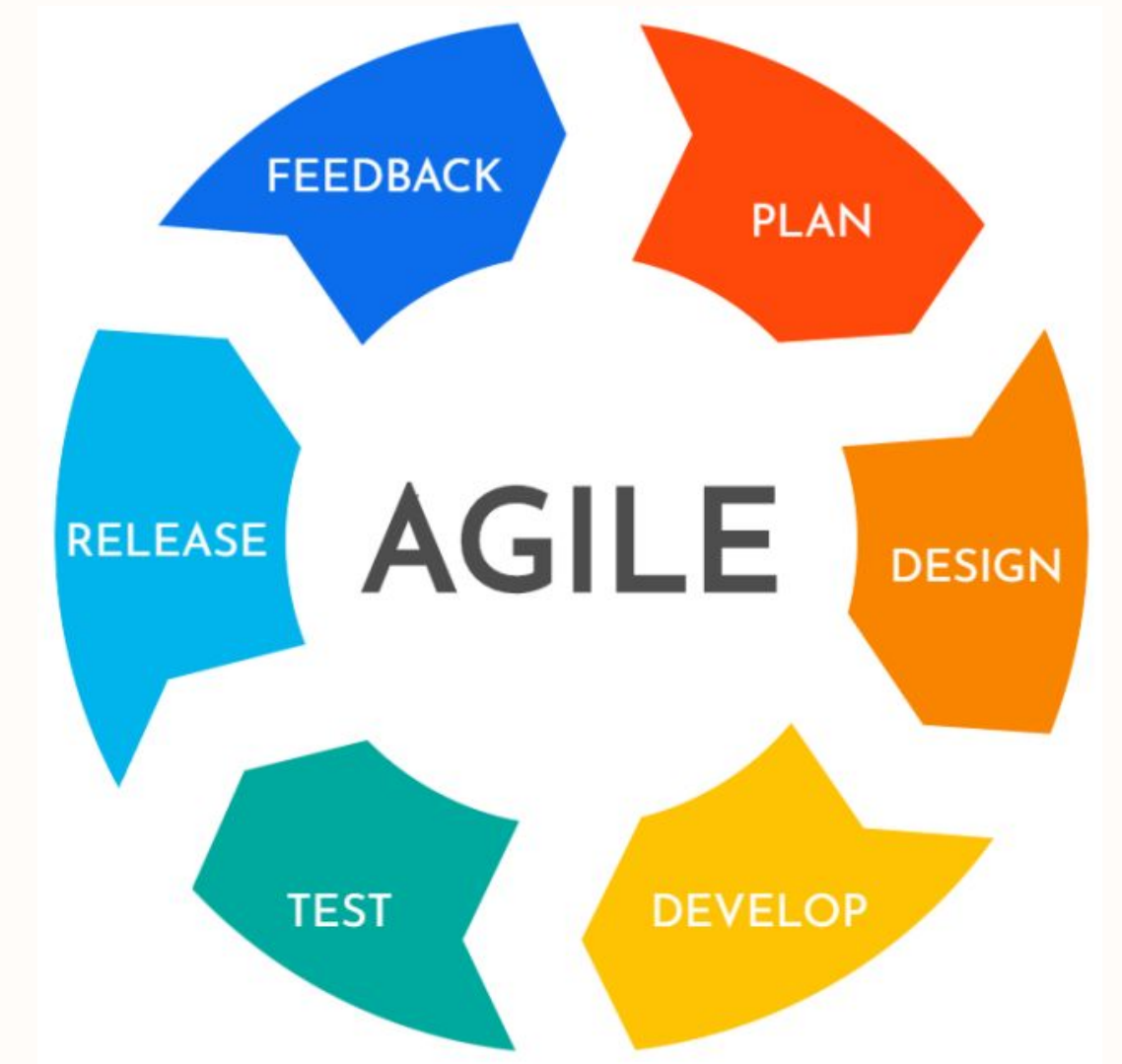
www.agilemanifesto.org

The Principles of Agile Methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

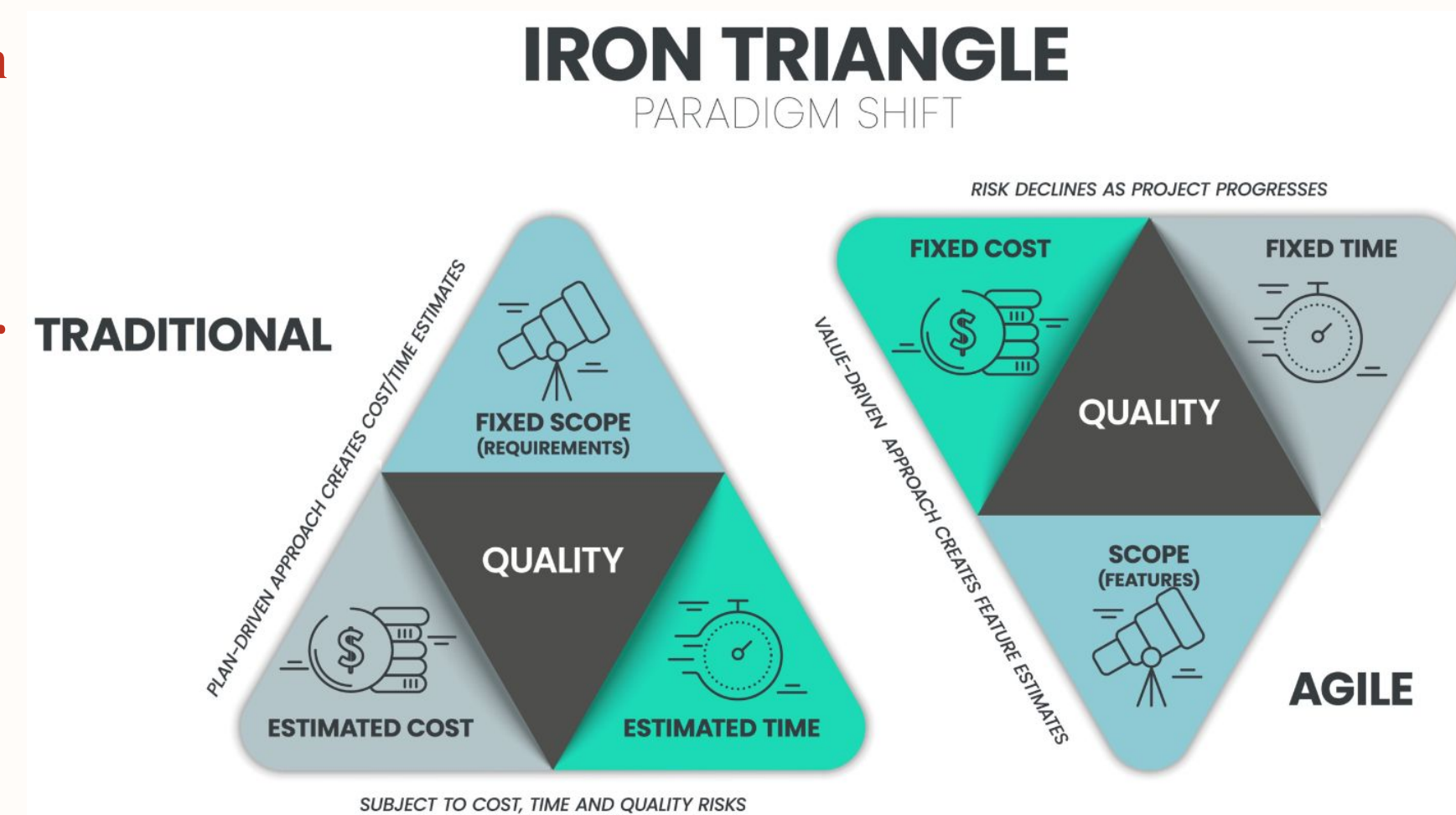
Agile Development

- **A time-boxed, iterative approach to software delivery that builds software incrementally from the start of the project**
- **Enables teams to deliver faster and respond to a change**
- **Most used agile methods:**
 - **SCRUM**
 - **KANBAN**
 - **Extreme Programming (XP)**
 - **Feature Driven Development (FDD)**
 - **Test Driven Development (TDD)**



Time-boxing

- Fixing the iteration end date and not allowing it to change
 - 1-6 weeks is the normal length for a time-boxed iteration
 - 3-6 months is usually too long and misses the point and value
- Shorter steps have lower complexity and risk, better feedback, and higher productivity and success rates
- The scope is reduced, rather than slip the iteration end date
- Lower priority requests back on the wish list
- Should not be used to pressure developers to work long hours to meet the deadline
 - If the normal pace of work is insufficient, do less



Benefits of Agile Development

- Customer satisfaction **by continuous delivery of software**
- **Working software is delivered frequently**
- **Greater flexibility and adaptability to change**
- **Increased collaboration frequency and feedback**
- **Close cooperation between stakeholders and developers**
- **Focused on Business Value**
- **Increased project control**

Problems and Challenges in Agile

- The project can easily get taken off track if the stakeholder is **not clear with what final outcome they want**
- It can be difficult to keep the interest of customers **who are involved in the process**
- The level of **collaboration can be difficult to maintain**
- The risk of losing **long-term vision as there is no clear end of the project**
- **Contracts may be a problem as with other approaches to iterative development.**

Problems and challenges in agile

- ✧ Documentation **tends to get sidetracked**
- ✧ Difficult **to measure progress**
- ✧ **Because of their focus on small, tightly-integrated teams, one needs to be careful when scaling agile methods to large systems.**
- ✧ Prioritizing changes **can be difficult where there are multiple stakeholders.**
- ✧ **Maintaining simplicity requires extra work**

1. Agile Practices

Agile Methods



Common points

- ✧ All Agile methods have these points in common:
 - Iterative **and** incremental **process**
 - Adaptive **planning**
 - **Effective** communication **and** stakeholder engagement
 - Short development **cycle allowing** regular delivery **of software**

User Story

- **The smallest unit of work in an agile framework**
- **An informal, natural language description of a feature or desired outcome of a software system**
- **Often written from the perspective of an end user of a system to influence the functionality of the system being developed**
- **May be written by various stakeholders including clients, users, managers, or development team members**
- **Related user stories can form epics**

As a < type of user >, I want < some goal > so that < some reason >.

As a **first-time visitor**, I want to **check out without creating an account** so that **I can complete my purchase faster.**

As a **user**, I want to **split a payment with friends via a shared link** so that **we can easily settle expenses.**

As a **customer**, I want to **receive SMS alerts for suspicious transactions** so that **I can act quickly if my card is compromised.**

As a **shopper**, I want to **filter search results by price range** so that **I can quickly find items within my budget.**

As a **user**, I want to **mute notifications for specific groups** so that **I avoid distractions during work.**

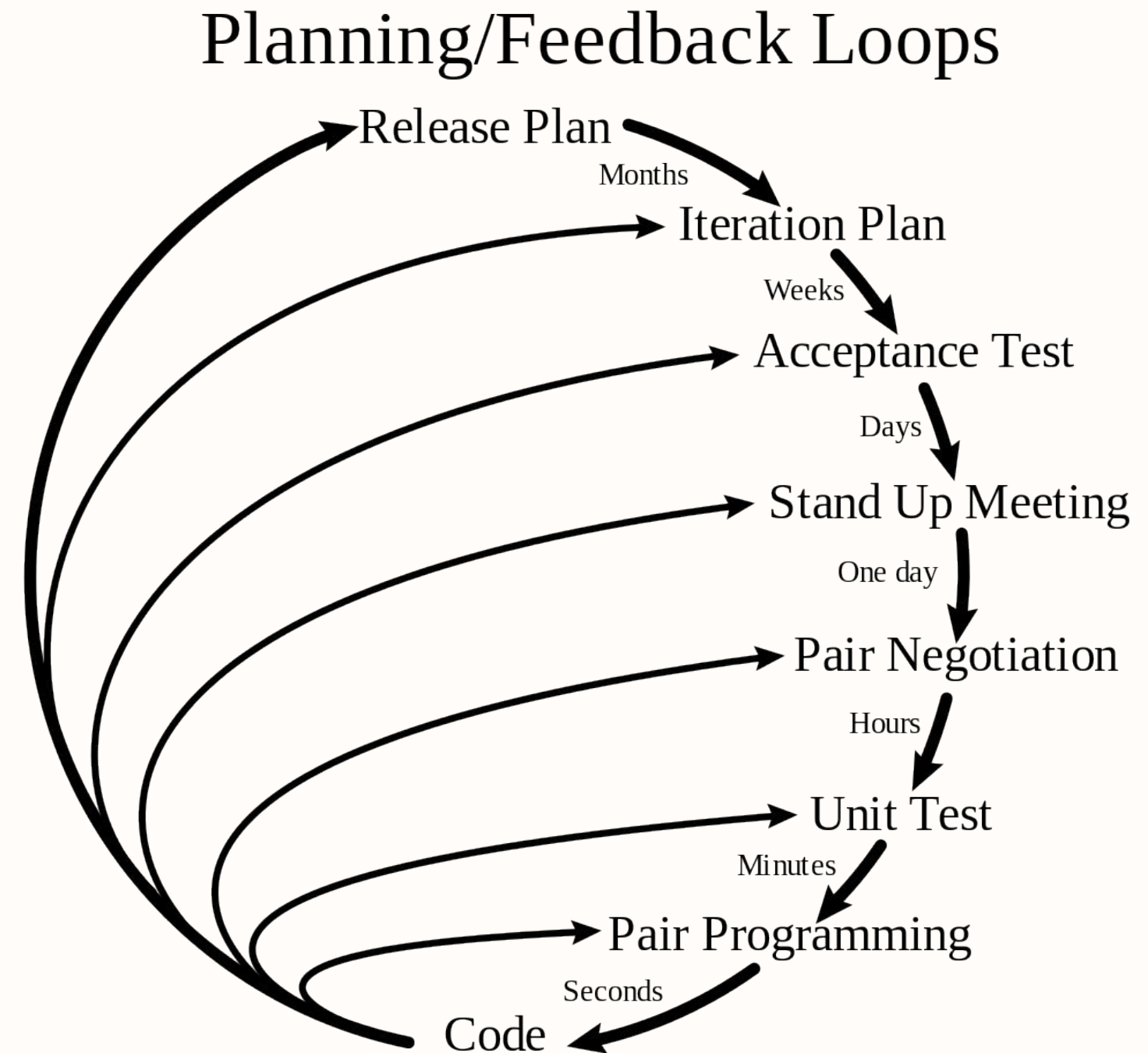
As a **patient**, I want to **get email reminders before my doctor's appointment** so that **I don't forget.**

As a **power user**, I want to **use keyboard shortcuts for common actions** so that **I can navigate faster.**

As a **night owl**, I want to **toggle on dark mode** so that **I can work comfortably in low light.**

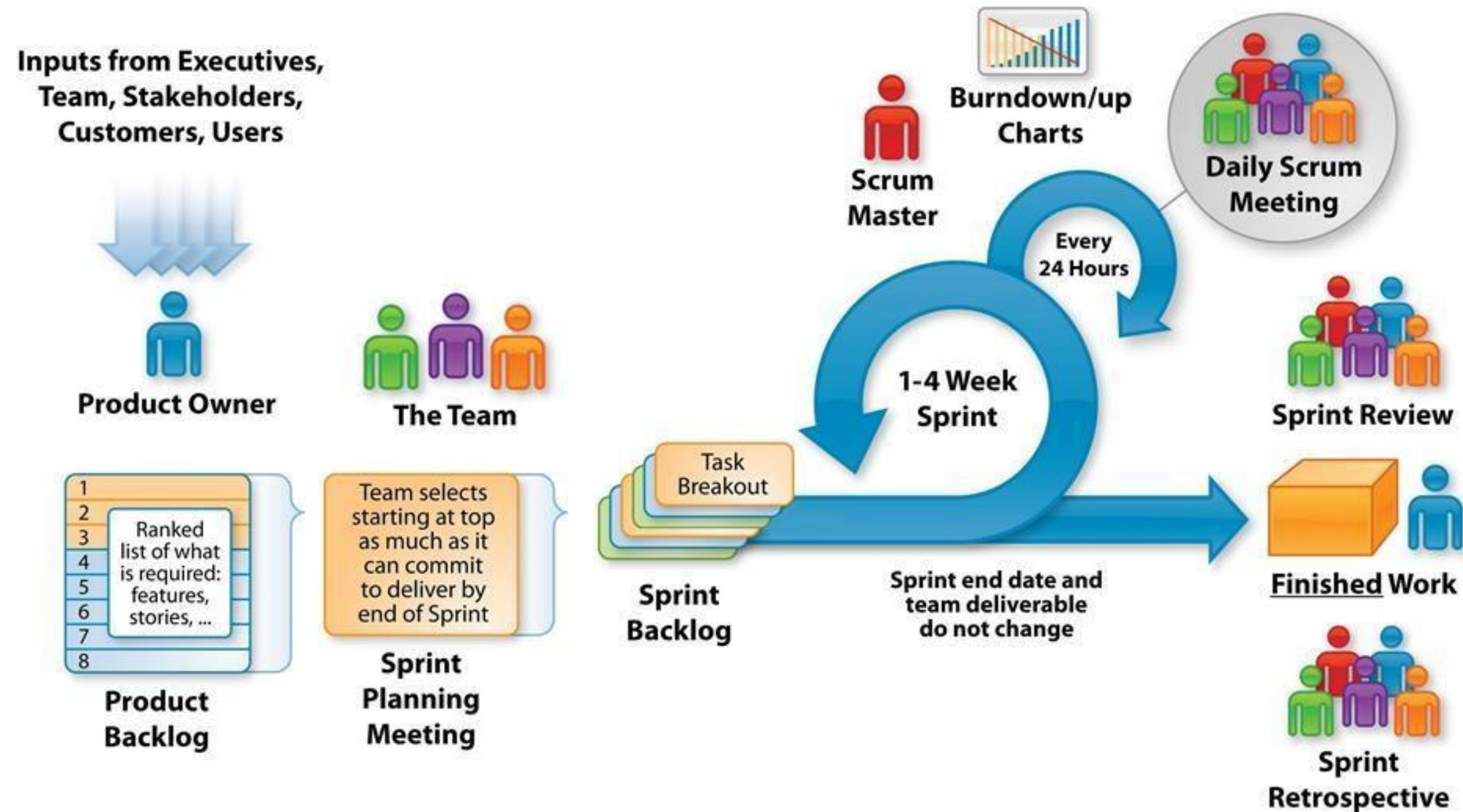
Extreme Programming

- An agile methodology designed to **improve the quality of software and its ability to adapt to the changing needs of the customer**
- **Iterative and frequent small releases**
- **Practices:**
 - Pair programming
 - Test driven development (TDD)



SCRUM

The Agile - Scrum Framework



SCRUM

The Agile - Scrum Framework

Inputs from Executives,
Team, Stakeholders,
Customers, Users

Product Owner

The Team



Product Backlog

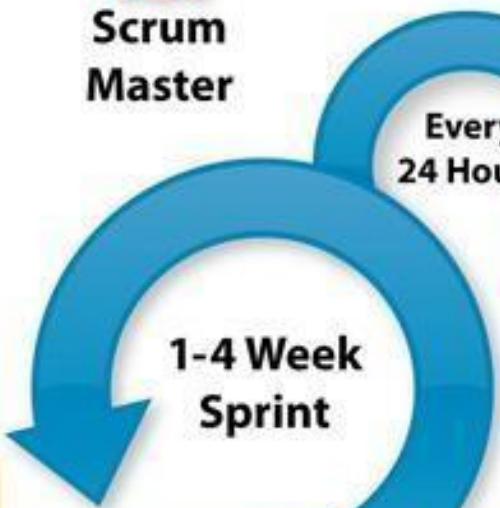
Team selects starting at top as much as it can commit to deliver by end of Sprint

Sprint Planning Meeting



Sprint Backlog

The list of tasks from product backlog to be completed by the development team during the next sprint



Scrum Master



Every 24 Hours



A 15-minute time-boxed event for the Development Team to synchronize activities and create a plan for the next 24 hours

Every team member should answer:

- What did I work on yesterday?
- What am I working on today?
- What issues are blocking me?



Sprint Review

The team reviews the work that was completed and plans the work that was not completed.

It involves demonstrating the working software to product owner and stakeholders



Finished Work

Sprint end date and team deliverable do not change



Sprint Retrospective

Team reflects on the past sprint and identifies and agrees on continuous process improvement actions.

The list of tasks to be done and contains a prioritized list of all product requirements that a team maintains for a product

Scrum Sprint 1

6 issues

8h

140h

0

Linked pages

0

...

Implement the new weather alert system 🌤️⚠️ - and make over 50,000+ customers...
02/Apr/18 1:21 PM • 13/Apr/18 1:21 PM



Recalibrate the semi-coherer	VERSION 1.0	SMART-43	↑	140h
Add app alert for changed weather events		SMART-17	↑	-
Update notifications settings with weather option		SMART-8	↓	5h
Push notifications documentation upc	Epic 456	SMART-3	↑	-
Low-power indicator optimasation on	Epic 123	SMART-6	↓	-
Investigate power outages		SMART-10	↑	3h

Backlog

16 issues

Create sprint

☒	Build the solar panel		SMART-12		
☒	Invert every graviton attractor		SMART-16		
	Update positronic circuits to amplify our multiphasic re		SMART-9		
☒	New control panel design		SMART-4		
☒	Account for antimatter modulatr		SMART-15		

SMART-17

🔊

👁️

⋮

Add app alert for changed weather events



Status

Done

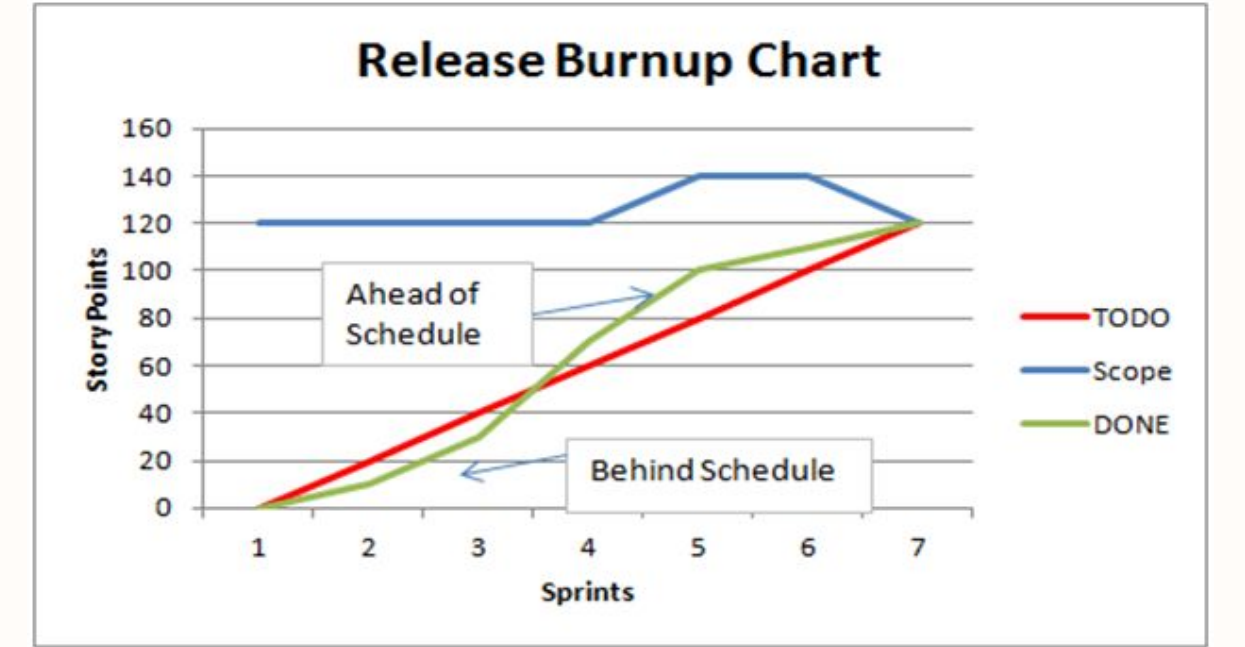
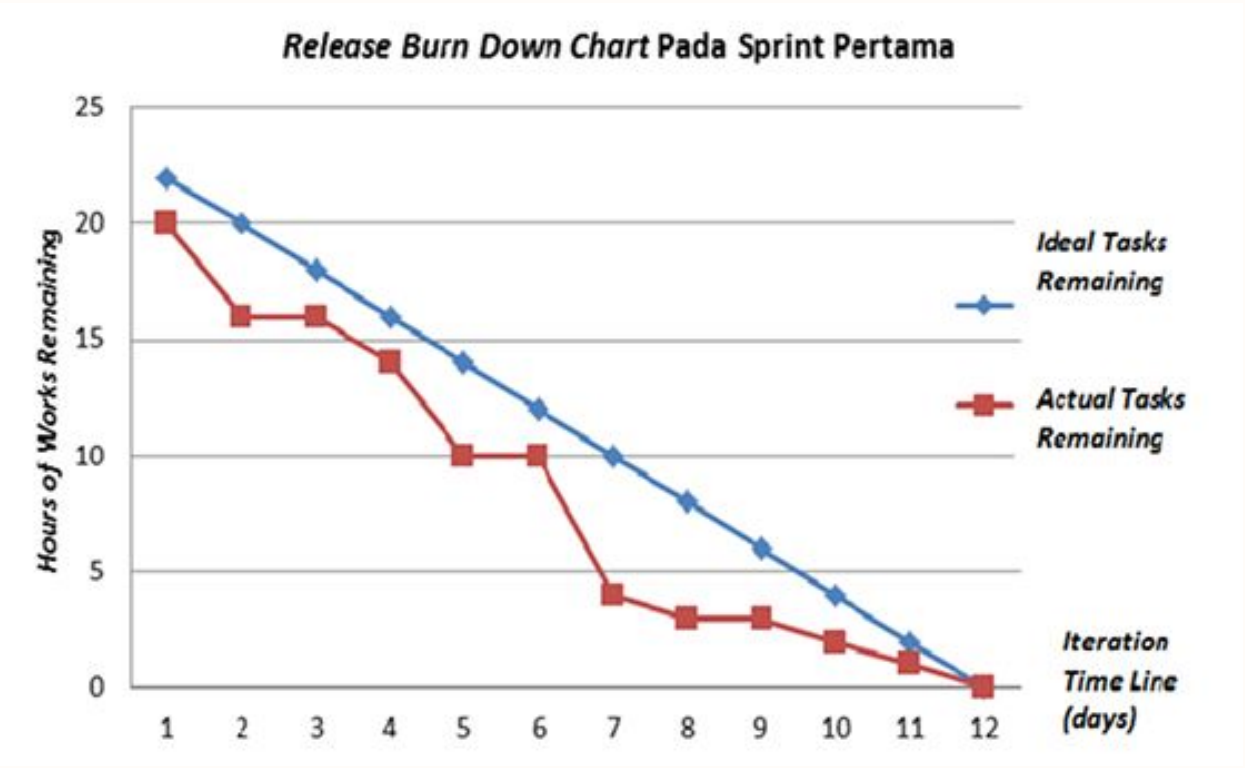
✓ Done

As a user I want to know when bad weather is approaching so I can cover or protect my solar panels.

- Scope & requirements
- Software change only
 - Third party weather tracking API
 - Does not include app alert development
 - Restore release notes

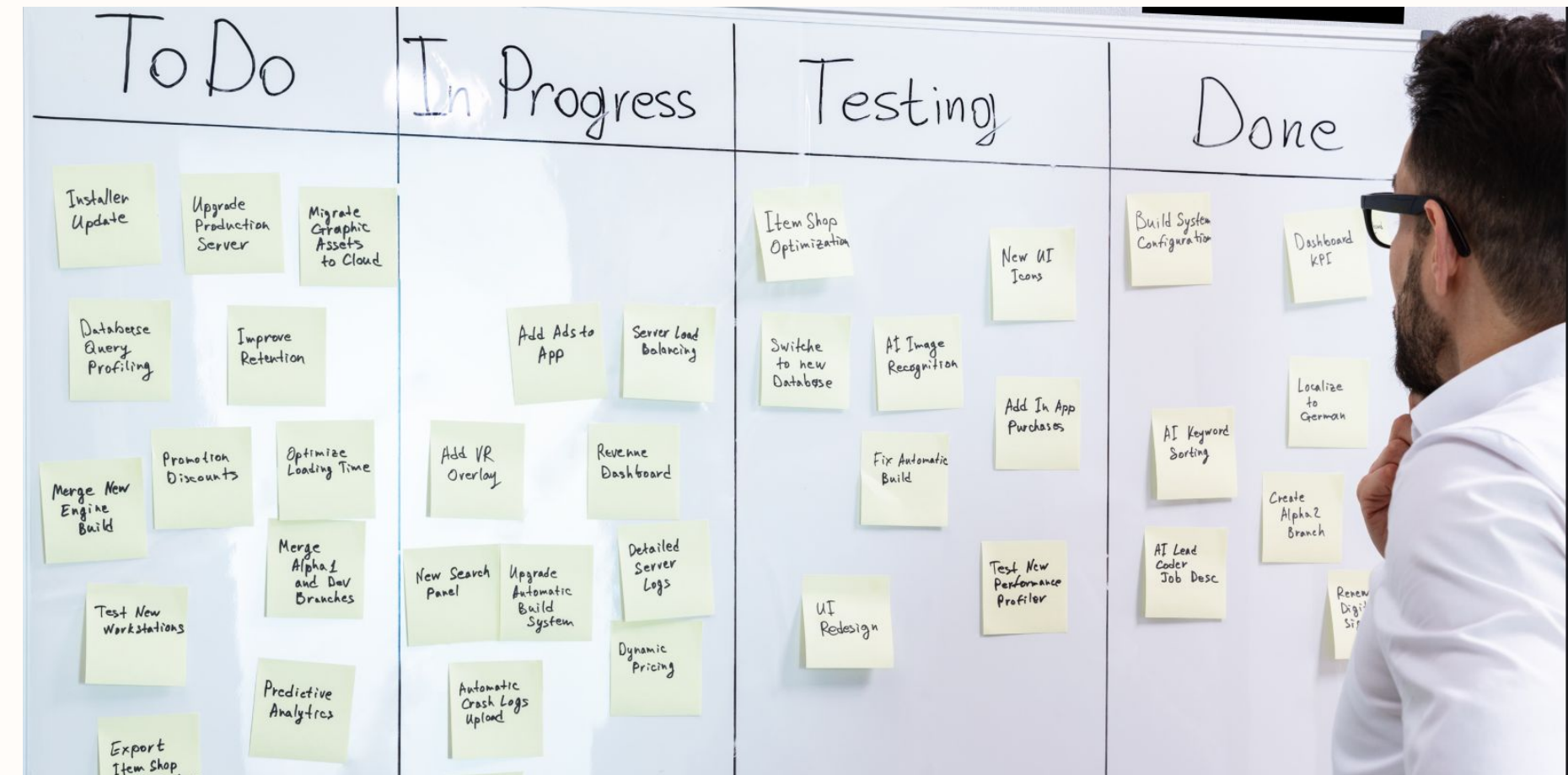
<http://google.com>

Attachments



Kanban

- A workflow designed to help visualize the work, maximize efficiency requiring real-time communication of capacity and full transparency of work
- Work items are represented visually on a kanban board, allowing team members to see the state of every piece of work at any time
- Skips the concept of iterations
- Relies on advanced tooling
 - continuous building, integration and deployment
- Two main practices are:
 - Visualize your work
 - Limit work-in-progress



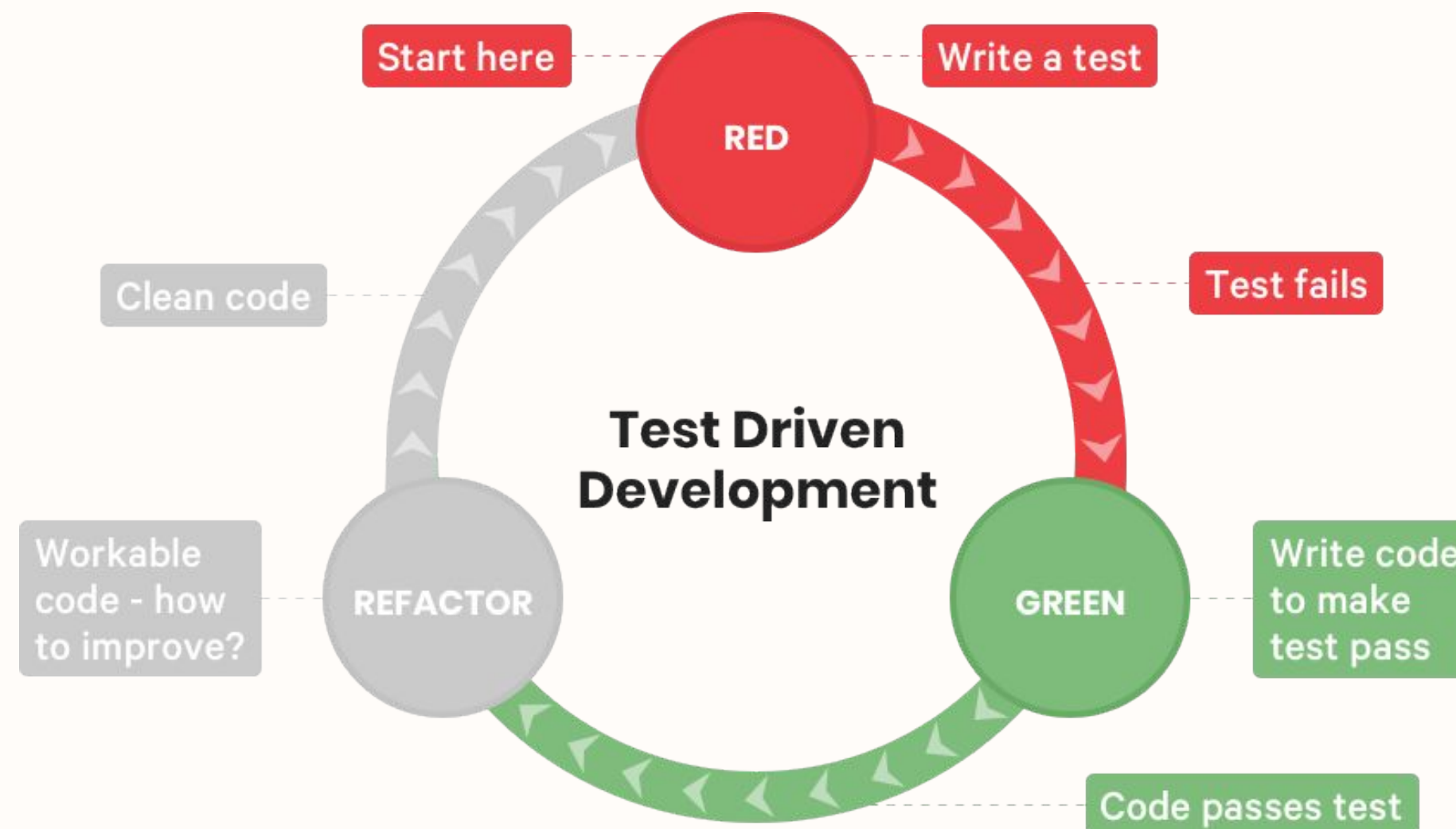
Feature-Driven Development

- A client-centered, architecture-centered, and pragmatic software process
- Ideal for long-term, complex projects looking for a simple but comprehensive methodology with clear outcomes
- Principles
 - Domain object modeling
 - Developing by feature
 - Individual class ownership
 - Feature teams
 - Inspections
 - Configuration management
 - Progress reports



Test-Driven Development

- A process that relies on the repetition of a very short development cycle: requirements are turned into very specific unit test cases, then the code is improved so that the tests pass



Agile Practices Summary

Project phases	Agile methodologies practices						
	XP	Scrum	ASD	FDD	DSDM	Crystal	TDD
Planning	Incremental design Spike Solutions	Sprint Planning	Adaptative cycle	Develop Overall Model	Study of business objective	Refine features	-
Requirements Analysis	CRC Cards User Story	Product Backlog Sprint Backlog	Mission declaration	Features list	User story	Vision document	-
Rules	10 minutes Build	2 - 4 weeks cycle	-	Development by features Regular Builds	Pareto principle 80%/20% Reversible changes	Fixed iterations Holistic Diversity Strategy	Work rested
Teams	Small teams Pairs Lead programmer	Small teams Multi-disciplinary	Multi-disciplinary	Features teams	Small teams	Several teams working in parallel	Solo Pairs Small teams
Codification	Refactoring Continuous integration Pair programming Collective code ownership	-	Technical review	Individual ownership code Inspections	Implementation of the prototypes	-	Pairing Refactoring Continuous integration
Estimatives	Planning games	Sprint planning	By mission	By Features	By Features	By Features	-
Meetings	Stand up meetings	Stand up meetings Sprint review	Analysis focused in customer	Domain Walkthrough	Business review	Workshop analysis	-
Monitoring	Project Velocity	Bumdown Chart Kanban	Milestones	Milestones	Milestones	Milestones	-
Tests	Unit tests Screening bugs	-	Integrated tests	Integrated tests	Integrated tests	Automated tests	Test first
Releases	Frequent	Frequent	Frequent	Frequent	Frequent	Frequent	Frequent